Linux网络----Udp协议(用户数据报协议)

UDP报头

16位源端口号 ----------16位目的端口号

16位Udp长度 ----------16位UDP校验和

----------|----------

8字节

|----------|

|　　正文　　|

|----------|

报头

16位源端口号：

> 发送者的端口号-----从绑定的ip的机器上的哪个进程发送的数据?

16位目的端口号：

> 接收者的端口号--用来标识进程

16位UDP校验和：

> UDP校验和的目的是为了验证传输的数据是否完整无误。在网络传输过程中，由于物理介质或网络设备的故障，数据中的某些比特位可能会发生翻转（即0变为1或1变为0），这种变化会破坏数据的完整性。UDP校验和通过计算并验证数据的完整性，帮助接收端检测并丢弃错误的数据包，从而确保数据的可靠性(即使udp不可靠)。

2个问题：

1.数据如何分离？

> 头部永远是八字节！！！只需要将message = 报头结构体数据 + 正文内容即可！！

2..如何交付？

> 将message的前8个字节直接提取出来--即头部字段，检验16位长度字段 是否 == 接收到的实际正文长度

Udp协议特性：

> 1.无连接
> 2.相较于TCP，UDP不可靠！
> 3.面向数据报 ----->发一次就接受一次，报文和报文之间边界明显！以数据块形式发送接收
> 换句话说，UDP基础下发送的信息一定是**原始的，**即发送端发送100个字节，接收端就必须一次性读到100个字节！

UDP缓冲区：

> UDP没有发送缓冲区！！直接将数据由sendto拷贝给内核，内核交付给网络层进行后续处理！！
> 但是UDP为了保证一定的可靠性，存在接收缓冲区--->
> 1.接收缓冲区如果已满，再次到来的数据直接被丢弃！
> 2.基于UDP协议的单次数据发送最大值为：64kb,如果超过这个阈值而不在应用层手动的进行分包的话，调用sendto会报错!

服务器使用UDP协议固定流程：

1.创建udp套接字--socket()

> socket()创建套接字

```c
#include <sys/types.h>          /* See NOTES */
#include <sys/socket.h>

int socket(int domain, int type, int protocol);
```

domain：协议族

> AF_INET---->ipv4
>
> AF_INET6--->ipv6

type：创建udp套接字还是tcp套接字？

> **SOCK_DGRAM---udp**
>
> **SOCK_STREAM---tcp**

protocol：

> 设置返回的描述符属性，设置为**SOCK_NONBLOCK**表示设置描述符非阻塞,设置**SOCK_CLOEXEC**表示不能被子进程继承

绑定---bind()

函数原型：

```c
#include <sys/types.h>          /* See NOTES */
#include <sys/socket.h>

int bind(int sockfd, const struct sockaddr *addr,
         socklen_t addrlen);
```

参数介绍：

> sockfd：不解释
>
> addr： **需要手动填充的服务器的地址信息**，sockaddr本质上c语言的“**拟基类**”，我们要填充的其实不是sockaddr，而是它的"子类"：struct sockaddr_in，结构体原型

```
sin_addr
sin_family
sin_port
sin_zero
```

CSDN @LIU_Skill

> 按顺序填充字段即可，zero为保留字段，无需关注。
>
> **sin_family**：==协议族
>
> **addrlen**：struct sockaddr_in的大小，sizeof可计算

直接通信

发送数据：sendto

```
ssize_t sendto(int sockfd, const void *buf, size_t len, int flags,
               const struct sockaddr *dest_addr, socklen_t addrlen);
```

接收数据：recvfrom

```
ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags,
                 struct sockaddr *src_addr, socklen_t *addrlen);
```

简单echo服务器server端：

```
#pragma once
#include <iostream>
#include <string>
#include <cstdlib>
#include <cstring>
#include <functional>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include "Log.hpp"
using namespace ns_log;
#define MAX_BUFFER_SIZE 4096
```

```
// Udp--无连接|不可靠|高性能|广播
// 云服务器bind--->0.0.0.0
/*这是一个echo服务器*/
class UdpServer
{
private:
    int _sockfd;
    uint16_t _port;
    bool _isrunning;
public:
    UdpServer(uint16_t port) : _port(port), _isrunning(false)
    {
        _sockfd = ::socket(AF_INET, SOCK_DGRAM, 0);
        if (_sockfd < 0)
        {
            LOG(FATAL, "create socketfd failed!\n");
            abort();
        }
        LOG(DEBUG, "---create sockfd succsee,sockfd = %d---\n", _sockfd);
        // bind
        struct sockaddr_in addr;
        memset(&addr, 0, sizeof(addr));
        addr.sin_family = AF_INET;
        addr.sin_port = ::htons(_port); // 本地序列转网络序列
        // bind任意ip
        addr.sin_addr.s_addr = INADDR_ANY;//INADDR_ANY--表示bind任意地址"0.0.0.0"
        int res = ::bind(_sockfd, (struct sockaddr *)&addr, sizeof(addr));
        if (res < 0)
        {
            LOG(FATAL, "server bind error!\n");
            abort();
        }
        LOG(DEBUG, "server bind success!\n");
    }
    void Start()
    {
        _isrunning = true;
        while (_isrunning)
        {
            char buff[MAX_BUFFER_SIZE];
            // 接收信息
            struct sockaddr_in peer;

            memset(&peer, 0, sizeof(peer));
            socklen_t len = sizeof(peer);
```

```
            ssize_t n = ::recvfrom(_sockfd, buff, sizeof(buff) - 1, 0, (struct sockaddr *)&peer, &len);
            if (n > 0)
            {
                buff[n] = 0;
                LOG(INFO, "client info#: %s\n", buff);
                // 发送
                std::string info = "[server][info]# ";
                info += buff;
                ssize_t n = ::sendto(_sockfd, info.c_str(), info.size(), 0, (struct sockaddr *)&peer, len);
            }
            else
            {
                if(n == 0) continue;//本次数据发收结束!
                if(n < 0)
                {
                    LOG(ERROR,"recv error!\n");
                    break;
                }
            }
        }
    }
    void Close()
    {
        if(_sockfd > 0) ::close(_sockfd);
    }
    void Stop()
    {
        _isrunning = false;
    }
    ~UdpServer()
    {
    }
};
```

Udp--done